

Homework #3 – Solutions

1. Decide which of the following identifier names is valid in C language. Support your decision.

_TOTal
float
Float
Total
123_total
total_int
total123

Answer: An identifier can be composed of letters (both uppercase and lowercase letters), digits and underscore '_' only, and the first letter of identifier should be either a letter or an underscore.

According to this definition of an identifier for C programming language, _TOTal, float, Float, Total, total_int and total123 are valid identifiers. However, 123_total is not.

2. Assume the following JavaScript program was interpreted using static-scoping rules. What value of x is displayed in function sub1? Under dynamic-scoping rules, what value of x is displayed in function sub1? Explain your answer.

```
var x;  
function sub1( ) {  
    document.write("x = " + x + "<br />");  
}  
function sub2( ){  
    var x;  
    x = 5;  
    sub1();  
}  
x = 15;    // Assume execution starts here  
sub2();
```

Answer: If static scoping rules are used, sub1() uses the global x which is equal to 15. So x= 15 is displayed on the screen.

If dynamic scoping rules are used, sub1() uses the x which is defined in sub2(), because sub2() calls sub1(). So, x= 5 is displayed on the screen.

3. This question is a programming exercise. In this question, you are asked to write three functions in C or C++: the first one declares a large array statically, the second one

declares the same large array on the stack, and the last one creates the same large array from the heap. Write some code to assign some initial values to array element, and do the same things in all three functions. Then, call each of the three functions a large number of times (at least 100000) and print the time required by function on the screen. Explain your results. You should also submit your source codes for this question.

Answer: For this experimental study, you need to write 3 C functions or programs.

In the first function, you need to define a global array like that:

```
#include "stdio.h"
#include "stdlib.h"
```

```
int A[100000]; // this is the global large array.
               // You may use larger arrays if you have enough memory.
```

```
void test1(){ // this function uses global (static) array
    int i;
    for (i=0; i<100000; i++)
        A[i]=i; // you may do some other array operations
}
```

```
void test2(){
    int B[100000]; // this function uses stack dynamic array
    int i;
    for (i=0; i<100000; i++)
        B[i]=i; // you may do some other array operations
}
```

```
void test3(){
    int *C; // this function uses heap dynamic array
    int i;
    C = (int *)malloc(100000*sizeof(int));
    for (i=0; i<100000; i++)
        *(C+i)=i; // you may do some other array operations
    free(C);
}
```

```
main(){
    int j;
    // get the system time here
    for (j=0; j<1000000; j++)
        test1();
```

```
// get the system time here, after the loop finishes
// compute the difference between two time values and print the result on the screen
```

```
// repeat the same process for test2() and test3() functions
}
```

Then, you should compare the time values printed on the screen. I expect that test1 is the fastest, test2 is the second fastest and test3 is the slowest function. The reasons for this result are as follows: the test1 function uses the static array, so it uses direct addressing and it does not have allocation and de-allocation costs. But test2() and test3() use indirect addressing, and have allocation/de-allocation costs. You may have different results if your array size is not big enough and if you do not call the functions large enough times. Also depending on your operating system, and other system calls you may get different results, but you should provide enough explanations for your results.

4. Assume the following rules of associativity and precedence for expressions:

Precedence	Highest	*, /, not +, -, &, mod Unary - =, !=, <, <=, >, >=
	Lowest	and or, xor
Associativity	Left-to-right	

Show the order of evaluation of the following expressions by parenthesizing all subexpressions and placing a superscript or subscript on the right parenthesis to indicate order.

$(a - b) / c$ and $(d * e / a - 3 + f = a)$

$a * b > c + d$ or $- 5 = z$

Answer: The orders of operations are as follows:

$((((a - b)_1 / c)_7 \text{ and } ((((((d * e)_2 / a)_3 - 3)_4 + f)_5 = a)_6)_8)$

$((((a * b)_1 > (c + d)_2)_4 \text{ or } ((- 5)_3 = z)_5)_6)$